

The Forecast Dashboard Book

A complete guide to understanding, using, and extending the interactive time-series predictor

Author: Calvin Okoth **Version:** 1.0 — Companion to the Forecast Dashboard (React + Vite)

Published: 2026-08-21

Table of Contents

1. [Introduction](#)
2. [Who This Book Is For](#)
3. [The Big Idea: What Forecasting Means](#)
4. [The Data the Model Understands](#)
5. [How a Forecast Is Produced](#)
6. [Model One — Linear Regression](#)
7. [Model Two — Moving Average](#)
8. [Model Three — Exponential Smoothing \(Holt\)](#)
9. [Model Four — Holt-Winters \(Seasonal\)](#)
10. [Confidence Intervals: How Sure Are We?](#)
11. [Reading the Graph](#)
12. [The Error Metrics \(MAE, RMSE, MAPE\)](#)
13. [Using the Dashboard](#)
14. [Limitations: What This Model Cannot Do](#)
15. [Under the Hood: Architecture & Code](#)
16. [A Worked Example](#)
17. [Running & Deploying the Dashboard](#)
18. [Glossary](#)

19. [Further Reading](#)
 20. [Backtesting: Proving the Forecast](#)
 21. [Model Comparison & Auto-Select](#)
 22. [Importing & Exporting Data \(CSV\)](#)
-

1. Introduction

The **Forecast Dashboard** is a small, self-contained web application that takes a list of numbers you paste in and predicts what comes next. It is an *interactive predictor*: you change the data or the settings, and the chart, the numbers, and the explanations update instantly.

This book explains, in plain language and with the underlying mathematics, exactly how that prediction is made. You do not need a background in statistics to read it — every formula is accompanied by a plain-English description. By the end you will understand:

- what kind of data the tool can read,
 - the four forecasting methods it offers and when to use each,
 - how it expresses uncertainty with prediction intervals,
 - how to read every element of the chart,
 - how to interpret the error scores,
 - and how the code itself is organized if you want to extend it.
-

2. Who This Book Is For

- **Curious beginners** who want to understand forecasting without heavy math.
- **Analysts** who need a quick, visual "what happens next" estimate from a raw series.
- **Students** learning time-series methods (regression, smoothing, seasonality).
- **Developers** who want to modify or extend the dashboard.

No single chapter requires the previous one, but reading them in order gives the smoothest path.

3. The Big Idea: What Forecasting Means

A **time series** is just a sequence of measurements taken one after another — daily sales, hourly temperature, monthly visitors, weekly weight. The position in the sequence is "time" (step 0, step 1, step 2, ...), and the value is what you measured.

Forecasting means: *look at the past, find a pattern, and assume the pattern continues into the future.* Every model in this dashboard does exactly that. They differ only in **what kind of pattern they assume**:

Assumption about the world	Model that matches it
"The future is a straight line from the past."	Linear Regression
"The future is roughly the recent average."	Moving Average
"The future follows the recent level and direction."	Holt (Exponential Smoothing)
"The future follows level, direction, and a repeating cycle."	Holt-Winters

None of them *know* the future. They make a reasoned guess based on the shape of what you provided.

4. The Data the Model Understands

The dashboard accepts a **single, univariate, numeric time series**. Let's unpack those words:

- **Single / univariate** — one number per time step. There are no extra columns (no "price" next to "date" next to "region").
- **Numeric** — plain numbers (integers or decimals).
- **Time series** — the order matters. The first number is step 0, the next is step 1, and so on.

How you enter it

You type or paste values separated by commas or spaces:

```
10, 12, 11, 14, 13, 16, 15, 18, 17, 20, 19, 22
```

The tool (`parseSeries` in `src/forecast.js`) splits that text on spaces and commas, converts each piece to a number, and drops anything that is not a valid number. Internally the series becomes a JavaScript array `number[]` , where the **array index is the time step**.

What it deliberately does *not* accept

- Dates or timestamps (time is just the integer position 0, 1, 2, ...).
- Labels, categories, or names.
- Multiple variables at once (no "predict sales from ad-spend").
- Missing values represented as blanks — they are simply ignored if they fail to parse.

Key point: because time is an index, the tool assumes your observations are **evenly spaced** (daily, monthly, hourly — whatever your unit is). It does not know the calendar.

Three **sample generators** let you experiment without typing:

- **Random walk** — a jittery, directionless path.
- **Trend** — a steady upward climb with noise.
- **Seasonal** — a sine wave (12-step cycle) plus noise — ideal for testing Holt-Winters.

5. How a Forecast Is Produced

Regardless of the method, the pipeline is always the same:

1. **Parse** the text into a numeric array `y[0...n-1]` .
2. **Fit** the chosen model to that history, producing a `fit` array (the model's best guess at each past point).
3. **Extrapolate** forward `horizon` steps to produce `forecast[0...H-1]` .
4. **Measure** the fit against the real history to compute error metrics.

5. **Construct** a 95% prediction interval around the forecast.

6. **Render** history, fit, forecast, and interval on one chart.

The central dispatch function is `forecast(series, options)` in `src/forecast.js`, which routes to the correct method based on the `method` option (`'linear'` , `'ma'` , `'holt'` , or `'holtwinters'`).

6. Model One — Linear Regression

Best when: the data shows a steady upward or downward trend and no repeating cycle.

The idea

Draw the best possible straight line through your points. "Best possible" means the line that minimizes the total squared distance from the points to the line (this is called **ordinary least squares**, or OLS).

The line has the form:

$$y = m \cdot x + b$$

- `x` is the time step (0, 1, 2, ...).
- `m` is the **slope** (how much `y` changes per step).
- `b` is the **intercept** (the value at step 0).

The math

Given `n` points, compute the means `$\bar{x}$` and `$\bar{y}$` . Then:

$$m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

$$b = \bar{y} - m \cdot \bar{x}$$

The **fit** at each step `i` is `$b + m \cdot i$` . The **forecast** `h` steps beyond the last observation (at index `n-1`) is:

```
forecast(h) = b + m * (n - 1 + h)
```

Because the formula is a straight line, the forecast is a straight continuation of the trend — it never curves or cycles.

In the code

`linearRegressionForecast(series, horizon)` in `src/forecast.js`. If the history is perfectly flat (denominator zero), the slope is set to 0 so it forecasts a constant.

7. Model Two — Moving Average

Best when: the series is roughly stable and noisy, with no clear trend or cycle.

The idea

Take the average of the **last window observations** and declare: "the future will stay around that average." Every forecast step is the same number.

The math

```
value = ( y[n-window] + ... + y[n-1] ) / window
forecast(h) = value      for all h
```

The **fit** is shown only for the most recent `window` points (the line is flat there). Earlier points have no fit (they are left blank on the chart).

Choosing the window

- **Larger window** → smoother, ignores recent bumps, but **lags** behind real changes.
- **Smaller window** → reacts fast to recent levels, but is **noisier**.

This model has no slope, so it cannot follow a trend — it will always flatten out.

In the code

`movingAverageForecast(series, horizon, window)` . The window is clamped to at least 1 and at most the number of data points.

8. Model Three — Exponential Smoothing (Holt)

Best when: the series has a level **and** a direction (trend), but no seasonality.

The idea

Holt's method keeps track of two evolving quantities:

- **Level** — roughly "where are we now?"
- **Trend** — roughly "which way and how fast are we moving?"

New observations gently pull the level and trend toward the latest data. Two smoothing weights control how quickly they adapt:

- **α (alpha)** — weight on the **level**.
- **β (beta)** — weight on the **trend**.

The math (simplified)

Each step updates:

```
level =  $\alpha \cdot y_t + (1-\alpha) \cdot (\text{level} + \text{trend})$     // blend new value with last forecast
trend =  $\beta \cdot (\text{level} - \text{prevLevel}) + (1-\beta) \cdot \text{trend}$ 
```

The **forecast** `h` steps ahead is:

```
forecast(h) = level + h · trend
```

So Holt can slope upward or downward, unlike Moving Average.

Reading the sliders

- **α and β near 1** → the model chases recent changes quickly (more wiggly, more reactive).

- **α and β near 0** → the model holds steadier, trusting its old estimates (smoother, slower).

In the code

```
holtForecast(series, horizon, alpha, beta) in src/forecast.js .
```

9. Model Four — Holt-Winters (Seasonal)

Best when: the series has a level, a trend, **and** a repeating cycle (e.g. monthly sales that rise every December).

The idea

Holt-Winters adds a third component — the **seasonal** pattern — on top of Holt's level and trend. The seasonal component captures "what usually happens at this point in the cycle."

You tell the model the **season length** m (for example, 12 for monthly data with a yearly cycle, or 7 for daily data with a weekly cycle). The model learns one seasonal adjustment per position in the cycle and repeats it into the future.

The math (additive form)

Three weights:

- **α (alpha)** — level
- **β (beta)** — trend
- **γ (gamma)** — season

Initialization (from the first cycles of history):

```
level0 = mean of first m points
trend0 = (mean of 2nd cycle - mean of 1st cycle) / m
seasonj = yj - level0          for j = 0 ... m-1
```

Then, stepping through the history, the model updates level, trend, and the seasonal term for the current cycle position $s = t \bmod m$. The forecast h steps ahead is:


```
forecast(h) = level + h·trend + season[(n-1+h) mod m]
```

This is why Holt-Winters can both *climb* (trend) and *wave* (season) at the same time.

Practical note

To estimate a season you need **at least about two full cycles** of history (roughly $2 \cdot m$ points). The code automatically clamps the season length to at most half the series length so it cannot ask for more cycles than you supplied.

In the code

```
holtWintersForecast(series, horizon, alpha, beta, gamma, seasonLength) in  
src/forecast.js .
```

10. Confidence Intervals: How Sure Are We?

A single forecast number is a *best guess*. The real future could be a bit higher or lower. The dashboard shows this uncertainty as a **95% prediction interval** — a band around the forecast.

The intuition

The model's past errors (its **residuals**) tell us how "surprising" new data tends to be. If the model was usually off by ± 5 in the past, the future is likely off by a similar amount — and the further we look ahead, the wider the uncertainty becomes.

The math

1. Compute residuals on the history: $e_i = y_i - \text{fit}_i$ (only where a fit exists).
2. Take their sample standard deviation σ (using $n-1$ in the denominator).
3. For forecast step h (0-indexed), the half-width is:

```
half(h) = 1.96 · σ · √(h + 1)
```

4. The interval is:

```
lower(h) = forecast(h) - half(h)
upper(h) = forecast(h) + half(h)
```

The factor **1.96** is the 95% critical value of the normal distribution; $\sqrt{(h+1)}$ makes the band **widen with the square root of the horizon**, so near-term steps are tight and distant steps are loose.

What it is *not*

This is a simple, approximate interval based on residual spread — it is not a full statistical forecast distribution, and it assumes the future error behavior resembles the past.

In the code

`confidenceIntervals(series, fit, forecast, z = 1.96)` in `src/forecast.js`. With fewer than two valid residuals it returns no interval.

11. Reading the Graph

The chart (`ForecastChart` in `src/components/ForecastChart.jsx`) shows everything on one timeline. Below the chart, the **"Reading this graph"** panel explains each element using your live data.

Element	Color	Meaning
History	Solid blue	The values you entered, from <code>t=0</code> to <code>t=n-1</code> .
Fit	Green dashed	The model's fitted line over the history (in-sample); basis for the error metrics.
Forecast	Orange dashed	The predicted values for the next <code>horizon</code> steps.
95% interval	Orange band	The expected range around the forecast, widening with the horizon.

Element	Color	Meaning
Boundary	Vertical grey dashed	The line at <code>t=n-1</code> : left = observed, right = predicted.

The history and forecast lines are joined at the last real point so the eye can follow the transition from "known" to "predicted."

12. The Error Metrics (MAE, RMSE, MAPE)

After fitting, the dashboard scores the model on how well its `fit` matched the actual history. These appear as cards at the top of the main panel.

- **MAE — Mean Absolute Error**

$$\text{MAE} = \text{average of } |\text{fit}_i - y_i|$$

The average size of the misses, in the original units. Easy to interpret.

- **RMSE — Root Mean Square Error**

$$\text{RMSE} = \sqrt{\text{average of } (\text{fit}_i - y_i)^2}$$

Like MAE but penalizes large errors more heavily. Use it when big misses matter most.

- **MAPE — Mean Absolute Percentage Error**

$$\text{MAPE} = \text{average of } |(\text{fit}_i - y_i) / y_i| \times 100\%$$

The error as a percentage of the actual values, so you can compare across scales.

Lower is better for all three. They describe **how well the model explained the past** — a useful but imperfect hint about how well it will predict the future.

In the code

`metrics(series, fit)` in `src/forecast.js`. Points without a valid fit (e.g. before the Moving Average window) are skipped.

13. Using the Dashboard

1. **Enter data** in the "Series" box, or click **Random walk / Trend / Seasonal** to load a sample.
2. **Pick a method** from the Model dropdown.
3. **Set the horizon** — how many steps into the future to predict (1–36).
4. **Adjust method-specific controls:**
 - Moving Average → **Window**.
 - Holt → **Alpha, Beta**.
 - Holt-Winters → **Season length, Alpha, Beta, Gamma**.
5. **Read the results:** the metrics cards, the chart (history + fit + forecast + interval), the "Forecast" chips (one value per step), the "How it works" explainer, and the "Reading this graph" panel.

Everything updates live as you change anything.

14. Limitations: What This Model Cannot Do

Be honest about the boundaries:

- **No seasonality** in the first three models — only Holt-Winters handles cycles.
- **No multivariate input** — a single series only; no external predictors.
- **No real timestamps** — time is an integer index, so irregular gaps and calendar effects are invisible.
- **No probabilistic rigor** — the 95% band is an approximation from residual spread, not a calibrated statistical interval.
- **Point forecasts only conceptually** — trend/season are extrapolated blindly; a Holt-Winters forecast will keep repeating the learned season forever, even if the real world

changes.

- **Sensitive to outliers and scale** — a single wild value can distort regression and smoothing.

Use it for quick, visual, single-series exploration — not for safety-critical or regulated forecasting.

15. Under the Hood: Architecture & Code

The project is a standard **React + Vite** single-page app with no backend. All forecasting happens in the browser.

```
forecast-dashboard/
├─ index.html
├─ package.json
├─ vite.config.js
└─ src/
    ├─ main.jsx           # React entry point
    ├─ App.jsx            # Dashboard UI, state, controls, explainers
    ├─ index.css          # Styling (dark theme)
    ├─ forecast.js        # ALL forecasting logic (pure functions)
    └─ components/
        └─ ForecastChart.jsx # Recharts line chart
```

The forecasting engine (src/forecast.js)

Pure, testable functions — no React, no DOM:

Function	Purpose
linearRegressionForecast	OLS straight-line fit + extrapolation
movingAverageForecast	Flat forecast from recent average
holtForecast	Level + trend exponential smoothing
holtWintersForecast	Level + trend + seasonal components

Function	Purpose
<code>confidenceIntervals</code>	95% band from residual σ
<code>forecast</code>	Dispatcher: picks a method
<code>metrics</code>	MAE, RMSE, MAPE
<code>backtest</code>	Holds out the last N points and scores the forecast
<code>compareModels</code>	Runs all four methods and ranks them
<code>parseSeries</code>	Text → <code>number[]</code>
<code>parseCSV</code>	CSV/file text → <code>number[]</code> (value column)
<code>sampleSeries</code>	Random / trend / seasonal generators

The UI (`src/App.jsx`)

Holds the state (text, method, horizon, sliders) with React `useState` / `useMemo` , builds the chart data, and renders:

- `Metric` — a single stat card.
- `ModelExplainer` — the "How it works" panel (method-specific text + interval note).
- `GraphExplainer` — the "Reading this graph" panel (live, data-keyed legend).
- `ForecastChart` — the visualization.

To extend the tool, the cleanest path is to add a new pure function in `forecast.js` , register it in the `forecast()` dispatcher, add an `<option>` in the method dropdown, and (if needed) add its controls in `App.jsx` .

16. A Worked Example

Load the **Seasonal** sample (a 12-step sine wave with noise, 48 points). Then:

1. Select **Holt-Winters**, set **Season length = 12**.
2. Try **Alpha = 0.5**, **Beta = 0.1**, **Gamma = 0.3**.

3. Set **Horizon = 24**.

You should see the green fit track the wave, the orange forecast continue both the upward drift (if any) and the repeating 12-step cycle, and the orange band widen as it projects further out. Switch to **Linear Regression** on the same data and notice the forecast becomes a single straight line that ignores the wave entirely — a clear illustration of "the model assumes the pattern you pick."

17. Running & Deploying the Dashboard

This book's companion app is a standard React + Vite project with **no backend** — all forecasting happens in the browser. You only need [Node.js](#) (v18+) installed.

Project layout

```
forecast-dashboard/  
├─ index.html  
├─ package.json          # dependencies and scripts  
├─ vite.config.js  
├─ BOOK.md               # this book  
└─ src/  
    ├─ main.jsx  
    ├─ App.jsx  
    ├─ index.css  
    ├─ forecast.js  
    └─ components/ForecastChart.jsx
```

Local development

From the project folder:

```
npm install      # install dependencies (first time only)  
npm run dev      # start the live dev server (hot-reload)
```

Vite prints a local URL such as `http://localhost:5173/`. Open it in your browser; every edit you save is reflected instantly.

Production build & preview

```
npm run build    # bundle the optimized static site into dist/  
npm run preview  # serve the built dist/ folder locally
```

The production build is a set of static files (HTML, CSS, JS) with no server logic, which makes it trivial to host anywhere static.

Publishing to the web (static hosts)

Because the output is static, you can deploy it to any of these without code changes:

- **Netlify / Vercel** — drag-and-drop the `dist/` folder, or connect the Git repository.
- **GitHub Pages** — push `dist/` to a `gh-pages` branch.
- **Any web server** — copy `dist/` to the server's web root.

Note: the app reads data you type and runs entirely client-side, so it needs no API keys, database, or server-side computation. "Deploying" simply means serving the static files.

Default dataset

On first load the dashboard shows this sample series so the chart is never empty:

```
10, 12, 11, 14, 13, 16, 15, 18, 17, 20, 19, 22
```

It is a mild upward trend — a good first thing to try each model against.

18. Glossary

- **Time series** — a sequence of values ordered in time.

- **Univariate** — a single variable measured at each step.
- **Horizon** — how many steps ahead to predict.
- **Fit** — the model's estimate of past values (in-sample).
- **Forecast** — the model's estimate of future values (out-of-sample).
- **Residual** — the error at a past point: `actual - fit`.
- **Level** — the current value the model thinks we are at.
- **Trend** — the direction and speed of change.
- **Season / Seasonality** — a repeating pattern of fixed length.
- **Smoothing weight (α , β , γ)** — how quickly a component adapts to new data (0 = slow, 1 = fast).
- **Prediction interval** — a range in which future values are expected to fall (here, 95%).
- **MAE / RMSE / MAPE** — error scores comparing fit to actuals.

19. Further Reading

- *Forecasting: Principles and Practice* (Hyndman & Athanasopoulos) — free online, the canonical intro to exponential smoothing and Holt-Winters.
- "Simple Linear Regression" — any introductory statistics text for the OLS formulas.
- "Moving Averages" — for baseline smoothing and trend removal.
- The **Recharts** documentation for how the visualization is rendered.
- The source files listed in Chapter 15 — the best documentation is the code itself.

20. Backtesting: Proving the Forecast

A model that fits the past beautifully can still fail on the future. **Backtesting** is the cure: temporarily hide the last `N` observations, ask the model to forecast them anyway, then measure how far off it was. It is the most honest accuracy check the dashboard offers, because it scores the model on data it never saw during fitting.

How to use it

In the **Model** panel there is a **Backtest holdout** slider (0 = off). Slide it to, say, 6 and the dashboard:

1. Trains the model on all but the last 6 points.
2. Forecasts those 6 withheld points.
3. Compares the forecast to the real values and reports **BT RMSE** and **BT MAE** in a panel.
4. Draws a **cyan dashed line** on the chart — the model's forecast over the withheld region — so you can eyeball it against the actuals (blue).

The math

With N held out, train on $y[0 \dots n-N-1]$, forecast $\hat{y}[n-N \dots n-1]$, and compute:

$$\begin{aligned} \text{BT RMSE} &= \sqrt{(1/N) \cdot \sum (\hat{y}_i - y_i)^2} \\ \text{BT MAE} &= (1/N) \cdot \sum |\hat{y}_i - y_i| \end{aligned}$$

Lower is better. A small backtest error is real evidence the model will generalize; a small *in-sample* error alone is not.

In the code

`backtest(series, options, testSize)` in `src/forecast.js`. It returns `{ testSize, test, pred, mae, rmse }`, or `null` if there is not enough history to train (it requires at least 5 training points).

21. Model Comparison & Auto-Select

Which of the four methods is *best* for your data right now? Instead of guessing, the dashboard can rank them all at once.

The comparison table

Below the chart is the **Model comparison** panel. It runs **Linear Regression, Moving Average, Holt, and Holt-Winters** on your current series and shows, for each:

- **In-sample RMSE** — error on the history it was fitted to.
- **Backtest RMSE** — error on the withheld holdout (when backtesting is on); otherwise a dash.

The methods are sorted by score (backtest RMSE if backtesting is enabled, otherwise in-sample RMSE), and the winner is marked with a star (★). Click **Use** to switch the active method to that winner.

Why it matters

Different shapes favor different models: a flat series loves Moving Average; a steady climb favors Linear or Holt; a wavy, repeating series favors Holt-Winters. The table turns that intuition into a measured, repeatable choice — and it updates live as you edit the data or the holdout size.

In the code

```
compareModels(series, options) in src/forecast.js returns an array of { method, label, inSampleRMSE, backtestRMSE, best }, already sorted with best set on the top row. METHOD_LABELS maps method keys to display names.
```

22. Importing & Exporting Data (CSV)

Typing numbers by hand is fine for a demo, but real work starts with files.

Import

In the **Data** panel, click **Import CSV**. Choose a `.csv` or `.txt` file. The parser (`parseCSV` in `src/forecast.js`) reads it line by line, takes the **last column** of each row (so a `date,value` file works), skips non-numeric header rows, and loads the values into the series. A plain comma- or space-separated list works too.

Export

Click **Export CSV** to download the current result as `forecast.csv` :

```
t,type,value,lower,upper
0,history,10
1,history,12
...
12,forecast,24.3,21.0,27.6
13,forecast,25.1,21.4,28.8
...
```

History rows have empty bounds; forecast rows include the **95% prediction interval** (lower/upper), so the file is ready for a spreadsheet or another tool.

In the code

Import uses the browser `FileReader` and `parseCSV` ; export builds a CSV string, wraps it in a `Blob` , and triggers a download via a temporary anchor element. Both live in `src/App.jsx` .

End of book. The dashboard and this guide are meant to be read together: change a control, watch the graph, and revisit the relevant chapter to understand why it moved.